2-oji praktinė užduotis: supaprastintos blokų grandinės kūrimas

Blokų grandinių technologijos | Individualiai arba poromis | 2026

1. Tikslas ir darbo apimtis

Sukurk programą, kuri generuoja vartotojus bei transakcijas, formuoja blokus, ieško darbo įrodymo (Proof-of-Work, PoW), tikrina duomenis ir atnaujina grandinės būseną. Panaudok 1-ajame darbe sukurtą maišos funkciją ir paaiškink, kaip ji susieja transakcijas bei blokus.

Tai **vietinė mokomoji simuliacija**, o ne Bitcoin protokolo realizacija. Pagrindinei užduočiai nereikia Bitcoin Core, regtest, tinklo ryšio, tikrų monetų ar skaitmeninių parašų. Penki kandidatiniai blokai v0.2 dalyje imituoja konkurenciją dėl kito bloko, tačiau savaime nesukuria decentralizuoto tinklo.

Vartotojai ir pradinė būsena -> transakcijų fondas -> kandidatinis blokas -> PoW -> patikra -> grandinės ir balansų atnaujinimas.

Versijos ir terminai

Terminai skaičiuojami nuo **2026-10-15**. v0.2 ir v0.3 turi tą patį galutinį terminą; v0.2 užfiksuok prieš pradėdamas v0.3 palyginimą.

Versija / terminas	Ką pateikti
v0.1 - 2026-10-29 (2 savaitės)	Veikianti vieno proceso grandinė, savo maiša, PoW, konsolės išvestis ir README.
v0.2 - 2026-11-05 (3 savaitės)	Merkle medis, išplėsta validacija, penki blokų kandidatai ir teisingumo testai.
v0.3 - 2026-11-05 (3 savaitės)	Dirbtinio intelekto (DI, angl. AI) įrankiais patobulinta versija ir patikrintas palyginimas su užfiksuota v0.2.

Kas lieka iš ankstesnio darbo?

Naudok savo deterministinę, fiksuoto ilgio maišos funkciją. Nurodyk jos repozitoriją ir versiją. Jei dirbate poroje, pasirinkite vieno iš jūsų algoritmą ir pagrįskite pasirinkimą. Programavimo kalba gali likti ta pati; taikyk aiškų atsakomybių atskyrimą, enkapsuliaciją ir kalbai tinkamą išteklių valdymą (C++ atveju - RAII).

Greita pateikimo patikra

[ ] Užfiksuotos v0.1, v0.2 ir v0.3 žymos / leidimai bei prasminga commit istorija.

[ ] Galima atkurti vartotojus, transakcijas, pradinę būseną ir testų rezultatus.

[ ] Balansai neneigiami, lėšų suma išsaugoma, ta pati transakcija neįtraukiama pakartotinai.

[ ] PoW ir transakcijų duomenys patikrinami prieš įtraukiant bloką.

[ ] README yra rezultatai, versijų palyginimas, DI naudojimo aprašymas ir narių indėliai.

Vertinimas: pagrindinė užduotis iki **2,0 balų**. Neprivalomai: neišleistų transakcijų išvesčių (UTXO) modelis **+0,5**, lygiagretus kasimas **+0,5**. Reikalavimai pateikti 6 skyriuje.

2. Duomenų modelis ir bendra eiga

Vartotojai ir transakcijos

Sugeneruok apie **1 000 vartotojų**, kiekvienam priskirdamas vardą, unikalų `public_key` ir pradinį balansą nuo **100 iki 1 000 000** mokomųjų vienetų. Pagrindinėje versijoje `public_key` gali būti tik imituojamas viešas identifikatorius: jis neįrodo rakto nuosavybės. Tikros kriptografinės raktų poros ir parašai neprivalomi.

Naudok **sąskaitų modelį**: būsena susieja vartotojo identifikatorių su balansu. Sumoms naudok sveikuosius skaičius, ne slankųjį kablelį. Pagrindiniame modelyje nėra komisinių ir kasimo atlygių, todėl bendra lėšų suma nekinta. UTXO alternatyva aprašyta 6 skyriuje.

Sugeneruok apie **10 000 transakcijų**. Kiekvienoje saugok `sender`, `receiver`, teigiamą `amount`, unikalų `sequence` ir `transaction_id`. Pastarąjį apskaičiuok iš visų kitų išvardytų laukų; paties ID į jo maišos įvestį neįtrauk. `sequence` leidžia atskirti du vienodos sumos mokėjimus tarp tų pačių vartotojų. Siuntėjas ir gavėjas turi egzistuoti ir būti skirtingi.

Bloko struktūra

Dalis / laukas	Paskirtis
Antraštė: `prev_hash`	Ankstesnio priimto bloko antraštės maiša.
Antraštė: `timestamp`, `version`	Laiko žyma (pvz., UTC Unix sekundės) ir duomenų formato versija.
Antraštė: `tx_root`	v0.1 - pagal bloko tvarką sujungtų transakcijų ID maiša; nuo v0.2 - Merkle šaknis.
Antraštė: `nonce`, `difficulty`	Keičiamas paieškos skaitiklis ir reikalaujamas pradinių hex nulių skaičius.
Turinys: transakcijos	Tikslia tvarka išsaugotas transakcijų sąrašas.

Genesis blokas: apibrėžk fiksuotą pradinį bloką, jo `prev_hash` (pvz., vien nuliai) ir tuščio turinio `tx_root` (pvz., H(tuščia baitų seka)). Genesis gali būti PoW išimtis, jei tai aiškiai dokumentuota. Pradinių balansų rinkinį išsaugok atskirai, kad būtų įmanoma atkurti būseną.

Vienas kartojamas ciklas

1. Iš fondo atsitiktinai, be pasikartojimų pasirink iki **100** tinkamų transakcijų. Paskutinis blokas gali būti mažesnis.
2. Jų taikymą tikrink laikinoje būsenos kopijoje, sąrašo tvarka. Kandidato kūrimas neturi keisti tikrųjų balansų.
3. Apskaičiuok `tx_root`, suformuok antraštę ir ieškok tinkamo `nonce`.
4. Iš naujo patikrink iškastą bloką. Tik tada pridėk jį prie grandinės, pritaikyk balansų pokyčius ir pašalink įtrauktas transakcijas iš fondo.
5. Kartok, kol fondas ištuštės arba neliks priimtinų transakcijų. Struktūriškai netinkamas atmesk su priežastimi; laikinai netinkamoms apibrėžk atidėjimo arba atmetimo taisyklę. Jų nelaikyk įtrauktomis ir nesuk nesibaigiančio ciklo.

3. v0.1: veikianti bazinė grandinė

Terminas: 2026-10-29. Įgyvendink visą 2 skyriaus eigą viename procese. v0.1 jau turi išsaugoti neneigiamus balansus ir bendrą lėšų sumą; v0.2 šias taisykles išplės atskiromis validavimo procedūromis ir neigiamais testais.

Nuoseklus kodavimas prieš maišą

Aprašyk laukų tvarką, teksto kodavimą ir skaičių užrašymą. Naudok vienareikšmį formatą, pavyzdžiui, laukų ilgius arba aiškiai apibrėžtą serializaciją. Vien sujungti `1` ir `23` nepakanka: be atskyrimo gaunami tie patys baitai kaip sujungus `12` ir `3`.

Paprastas sprendimas: naudok fiksuotą laukų tvarką `sender|receiver|amount|sequence`. Pavyzdžiui, sender = `a12f`, receiver = `b34e`, amount = `100`, sequence = `5` serializuojami kaip `a12f|b34e|100|5`. Tai sutrumpinti mokomieji identifikatoriai. Transakcijos ID yra šio teksto UTF-8 baitų maiša, įskaitant skirtukus; paties ID į įvestį nededame.

Skirtukas nepriklauso laukų reikšmėms. Hex identifikatoriai turi tik 0-9 ir a-f, todėl `|` juose nepasitaiko. Skaitines sumas ir sequence rašyk dešimtaine forma be tarpų, nereikalingų pradinių nulių ar pliuso. Nepridėk eilutės pabaigos. Laisvo teksto laukams, kuriuose gali būti `|`, reikėtų sutarto ekranavimo arba ilgio baitais prefikso. Kūrimui ir patikrai naudok tą pačią serializavimo funkciją.

Maišos išvestis turi būti vienodo ilgio ir išsaugoti pradinius nulius. Bloko hash skaičiuok iš **šešių antraštės laukų**, ne iš atmintyje esančio objekto adreso ar priklausomo nuo aplinkos jo atvaizdavimo. Turinį su antrašte susieja `tx_root`.

Darbo įrodymas

- Pagrindiniam bandymui naudok **difficulty = 3**: antraštės maišos hex užrašas turi prasidėti bent `000`.
- `nonce` nebūtinai atsitiktinis: gali jį nuosekliai didinti. Vieno kandidato paieškoje kitus antraštės laukus laikyk fiksuotus.
- Nekeisk gautos maišos ir neprirašyk nulių vien tam, kad ji atitiktų sąlygą. Jei savo maišą tenka patobulinti, užfiksuok pakeitimą ir pakartok jos formato bei determinizmo testus.
- PoW tikrinimas turi iš naujo apskaičiuoti maišą ir palyginti ją su deklaruota sąlyga. Vien išsaugoto hash prefikso patikros nepakanka.

Tai mokomasis sudėtingumo apibrėžimas. Bitcoin lygina antraštės maišą su skaitiniu target; šio protokolo ar jo sudėtingumo perskaičiavimo realizuoti nereikia.

Matoma programos eiga

Konsolėje pateik glaustą kiekvieno priimto bloko suvestinę: aukštį, transakcijų skaičių, `prev_hash`, `tx_root`, bloko hash, `difficulty`, rastą `nonce`, bandymų skaičių ir kasimo laiką. Neprivaloma spausdinti kiekvieno nesėkmingo bandymo.

Pabaigoje parodyk vartotojų ir transakcijų skaičius, priimtus blokus, įtrauktas / atmestas / likusias transakcijas, pradinę ir galutinę lėšų sumas bei bendrą laiką. Turi būti įmanoma paaiškinti kiekvieną likusią ar atmestą transakciją.

Ką patikrinti prieš v0.1 pateikimą?

- Mažas rankomis patikrinamas atvejis su keliais vartotojais ir mokėjimais.
- Tas pats įvesties rinkinys duoda tas pačias transakcijų maišas; fiksuota antraštė duoda tą patį bloko hash.
- Vartotojas negali išleisti daugiau, nei turi, įskaitant kelis jo mokėjimus tame pačiame bloke.
- Veikia paskutinis nepilnas blokas, nėra dubliuotų transakcijų ir neigiamų balansų.

Pateik: v0.1 leidimą ir README su paleidimu, modelio aprašymu, maišos versija, generavimo seed, bent vieno bloko išvestimi ir galutine suvestine. Pradinę būseną išsaugok prieš taikydamas transakcijas.

4. v0.2: Merkle medis ir validacija

Terminas: 2026-11-05. Išlaikyk v0.1 veikimą ir papildyk jį šiomis galimybėmis.

Merkle medis

Lapai - transakcijų ID jų tvarka bloke. Vidinį mazgą skaičiuok kaip H(kairiojo vaiko baitai || dešiniojo vaiko baitai), naudodamas savo maišą. Hex ID prieš jungimą dekoduok į baitus. Kai lygyje lieka neporinis mazgas, poruok jį su jo kopija. Jei yra vienas lapas, jis yra šaknis; tuščio genesis atvejį apibrėžk atskirai.

Patikrink atvejus su **1, 2, 3 ir 5 transakcijomis**. Parodyk, kad pakeitus transakcijos turinį bei perskaičiavus jos ID arba sukeitus skirtingų transakcijų tvarką, šaknis paprastai pasikeičia. Jei savo maišai aptinki koliziją, ją dokumentuok, o ne slėpk.

Transakcijos ir bloko tikrinimas

- **Transakcija:** žinomi dalyviai, teigiama sveikoji suma, tinkamas ID, nėra jau įtraukto ar bloke pasikartojančio ID. Patikrink balanso pakankamumą ir skaičių perpildymą.
- **Eiliškumas:** visas kandidatas tikrinamas jo transakcijų tvarka vienoje laikinoje būsenoje. Jei A turi 100, du mokėjimai po 70 negali abu būti priimti vien todėl, kad kiekvienas atskirai mažesnis už 100.
- **Blokas:** tinkamas `prev_hash`, palaikoma formato versija, teisinga `tx_root`, leidžiamas transakcijų skaičius ir PoW pagal grandinės nustatytą sudėtingumą. Kandidatas negali pats sumažinti reikalaujamo sudėtingumo.
- **Atmetimas:** netinkamas blokas nekeičia grandinės, tikrųjų balansų ar įtrauktų transakcijų aibės. ID sutapimas tikrina duomenų vientisumą, tačiau neatstoja siuntėjo skaitmeninio parašo.

Penki kandidatai kitam blokui

1. Tam pačiam grandinės aukščiui sudaryk **5 kandidatus**, kiekviename iki 100 transakcijų. Jie remiasi tuo pačiu `prev_hash` ir pradine būsena. Kandidatų transakcijos gali persidengti, bet viename kandidate nesikartoja.
2. Kiekvienam skirk ribotą bandymų skaičių arba laiką, pavyzdžiui, 5 s. v0.2 pakanka nuoseklaus vykdymo; užrašyk kandidatų tikrinimo tvarką ir pasirinktą biudžetą.
3. Jei sprendimo nėra, padidink biudžetą ir tęsk paiešką nuo dar netikrintų nonce. Nemažink sudėtingumo. Numatyk bendrą ribą arba aiškų sustabdymą, jei paieška užtrunka.
4. Pagal dokumentuotą taisyklę pasirink vieną tinkamą laimėtoją ir jį iš naujo patikrink. Įtrauk tik jo transakcijas. Nelaimėjusių kandidatų poveikio balansams nėra; likusiems mokėjimams kandidatus sudaryk iš naujo.

Pateik: Merkle ir validavimo testus (bendras rinkinys - 5 skyriuje), kandidatų bandymų / laiko suvestinę ir README paaiškinimą, kodėl ši konkurencijos simuliacija dar nėra P2P konsensusas. Nereikalaujama tinklo, šakų persitvarkymų ar tikro Bitcoin kasimo.

Transakcijų tikrinimas (4 skyriaus tęsinys)

Vienkartinė patikra prieš įtraukiant į transakcijų fondą (pool) negarantuoja, kad transakciją bus galima įvykdyti pasirinktame bloke. Atskirai įgyvendink pirminę patikrą, kandidato tikrinimą ir galutinį bloko priėmimą.

1. Prieš priimant į pool

Patikrink laukų formatą, dalyvių egzistavimą, teigiamą sveikąją sumą ir perskaičiuotą transaction_id. Nepriimk ID, kuris jau yra fonde ar patvirtintoje grandinėje. Galima tikrinti ir dabartinį balansą, tačiau šis rezultatas galioja tik tos būsenos atžvilgiu. Vien patekimas į pool nerezervuoja siuntėjo lėšų.

2. Atrenkant transakcijas į kandidatą

Nukopijuok patvirtintus balansus į laikiną būseną. Transakcijas tikrink ir pritaikyk šiai kopijai jų numatoma tvarka bloke. Atrink iki 100 transakcijų, kurias taip galima nuosekliai įvykdyti. Kiekvienas iš penkių kandidatų turi atskirą kopiją, pradedamą nuo tos pačios patvirtintos būsenos.

Pavyzdys: A turi 100 vienetų. Fonde yra T1: A -> B, 70 ir T2: A -> C, 60. Abi atskirai praeitų balanso patikrą pagal pradinį 100 balansą, tačiau kartu reikalauja 130.

Žingsnis	Kandidato laikina būsena
Tikriname T1	70 <= 100: įtraukiame T1, laikinas A balansas tampa 30.
Tikriname T2	60 > 30: T2 į šį kandidatą neįtraukiame.
Jei pirmiausia pasirinkta T2	A liktų 40, todėl T1 šiame kandidate nebetiktų.

Tai nėra vienodos transakcijos ar dubliuotas ID. Problema - bendras lėšų poreikis. Tikrieji balansai atrankos metu lieka nepakeisti. Nelaimėjęs kandidatas neturi jokio poveikio patvirtintai būsenai.

3. Prieš priimant iškastą bloką

Iš naujo patikrink transakcijas jų tvarka pagal tuometinę patvirtintą būseną, ankstesnio bloko nuorodą, Merkle šaknį ir PoW. Tik visam blokui praėjus patikrą, vienu nuosekliu veiksmu atnaujink grandinę bei balansus ir pašalink įtrauktas transakcijas iš pool. Jei blokas netinkamas, nieko iš jo nepritaikyk.

Atmesti ar atidėti?

Neteisingas ID, neigiama suma ar nežinomas dalyvis yra priežastis atmesti. Nepakankamas balansas gali būti laikinas: siuntėjas vėliau gali gauti lėšų. Paprastesnėje realizacijoje tokią transakciją galima atmesti su aiškia priežastimi; pasirinkus atidėjimą, ją palik fonde ir patikrink pasikeitus būsenai. Jei visas atrankos ciklas nesukuria pažangos, sustok ir pateik likusių transakcijų suvestinę.

sequence šioje užduotyje: pakanka bendro unikalaus generatoriaus numerio. Jei papildomai realizuoji atskirą griežtą siuntėjo skaitiklį, tikrink jį laikinoje būsenoje kartu su balansu. Dar neatėjusio numerio transakciją atidėk; mažesnį jau panaudotą numerį atmesk. Vien numerio pakeitimas neįrodo siuntėjo leidimo mokėti.

5. v0.3: patobulinimas su AI / DI įrankiais

Terminas: 2026-11-05. Užfiksuok veikiančią v0.2 kaip nekintamą atskaitos versiją. Tada naudok pasirinktą DI įrankį kodui peržiūrėti ir patobulinti. v0.3 turi išlaikyti ankstesnius reikalavimus; DI pasiūlymas nėra teisingumo įrodymas.

Darbo eiga su DI

1. Pasirink **bent dvi konkrečias tobulinimo kryptis**: validavimo klaidos, testų aprėptis, kodo struktūra, atminties naudojimas, kasimo ciklo ar Merkle medžio skaičiavimo efektyvumas.
2. Pateik įrankiui problemą, esamus reikalavimus ir aktualų kodo fragmentą. Išsaugok įrankio / modelio pavadinimą, datą ir reikšmingas užklausas arba glaustą sąveikos žurnalą.
3. Įvertink pasiūlymus. Priimtus pakeitimus fiksuok atskirais commits; bent vienam pasiūlymui aprašyk, ką ir kaip patikrinai. Paaiškink atmestus pasiūlymus, jei tokių buvo.
4. Paleisk tuos pačius teisingumo testus v0.2 ir v0.3. Palyginimui atkurk tą pačią pradinę būseną ir naudok tuos pačius duomenis bei kasimo sudėtingumą.
5. Pateik išvadą: kas pagerėjo, kas nepasikeitė ar pablogėjo, ir kokie apribojimai liko. Jei pasiūlymas naudos nedavė, pateik bandymą bei įrodymus; nereikia išgalvoti pagerėjimo.

DI pagalbą ankstesniuose etapuose taip pat atskleisk. v0.3 palyginimo atskaita yra užfiksuota v0.2; jos nevadink „be DI“, jei DI jau buvo naudotas. Kiekvienas autorius turi gebėti paaiškinti priimtus pakeitimus.

Bendras teisingumo testų rinkinys

Scenarijus	Laukiama savybė
Galiojantis mokėjimas; galutinis blokas su <100 įrašų	Teisingi balansai, lėšų suma nekinta, mokėjimas įtrauktas vieną kartą.
Nežinomas gavėjas; suma 0 ar neigiama; pakeistas ID	Transakcija atmetama, priežastis matoma.
Du to paties siuntėjo mokėjimai viršija balansą	Bendrai neleistinas rinkinys nepriimamas; laikomasi vykdymo tvarkos.
Dubliuotas ID arba jau patvirtinto mokėjimo pakartojimas	Nėra pakartotinio įskaitymo.
Pakeistas bloko turinys, prev_hash arba netinkamas PoW	Blokas atmetamas, patvirtinta būsena nepasikeičia.
Merkle atvejai 1 / 2 / 3 / 5 ir nelaimėję kandidatai	Teisinga medžio konstrukcija; kandidatų būsenos nesusimaišo.

Kaip pagrįsti spartos teiginį?

Jei teigi, kad programa pagreitėjo, matuok **bent 5 kartus**, pateik vidurkį ir sklaidą. Atskirk skaičiavimus nuo konsolės išvedimo. Naudok tą pačią mašiną, kompiliavimo parinktis, maišos versiją ir fiksuotas kandidatų antraštes; fiksuoto bandymų kiekio sparta palyginama patikimiau nei vien laikas iki atsitiktinai rasto sprendimo. Jei pakeitei pačią maišą, jos rezultatų nemaišyk su vien kodo optimizavimo palyginimu.

6. Pateikimas, vertinimas ir papildymai

Repozitorija ir README

Pateik viešos GitHub repozitorijos nuorodą pagal kurso pateikimo tvarką. Joje turi būti pažymėtos **v0.1, v0.2 ir v0.3** versijos, prasminga commit istorija ir paleidimo instrukcijos. Prie v0.3 išsaugok v0.2 palyginimo atskaitą. Slaptų prisijungimo duomenų ar DI API raktų nekelk.

README turi apimti:

- **Paleidimą:** kalbą, priklausomybes, kompiliavimą, mažo pavyzdžio ir viso rinkinio vykdymą, testų komandą.
- **Modelį:** vartotojų / transakcijų / bloko laukus, serializaciją, maišos versiją, genesis, balansų bei netinkamų transakcijų tvarkymo taisykles.
- **Atkuriamumą:** seed, generatorių ir jo aplinką arba išsaugotus duomenis, pradinius balansus, sudėtingumą, kandidatų biudžetus ir matavimo sąlygas.
- **Įrodymus:** bloko pavyzdį, galutinę suvestinę, testų lentelę ir v0.2 / v0.3 palyginimą. Ekranvaizdžiai turi papildyti, o ne pakeisti patikrinamus rezultatus.
- **DI naudojimą:** įrankius / modelius, užklausų žurnalą, priimtus ar atmestus pasiūlymus ir jų patikrą. Nurodyk naudotus šaltinius bei kitų autorių kodą.

Vertinimas: iki 2,0 balų

Individualiai ir poromis taikomi tie patys baziniai reikalavimai: nuosekli ir laiku užfiksuota projekto raida; duomenų, PoW ir validavimo logikos teisingumas; aiški kodo struktūra; atkuriami testai; suprantama demonstracija ir README; pagrįstas v0.3 palyginimas; savarankiškas supratimas ir plagijavimo nebuvimas.

Dirbant poromis: kuriamas vienas bendras projektas. Abiejų indėlis turi būti matomas commit istorijoje ir README. Galite pasidalyti kasimą, validavimą, Merkle medį ar testus, tačiau abu turite gebėti paaiškinti bendrą eigą ir savo pakeitimus. DI naudojimas nepakeičia autoriaus atsakomybės.

Neprivalomi papildomi balai

Užduotis	Reikalavimai
UTXO modelis: +0,5	Vietoje pagrindinio sąskaitų modelio realizuok įvestis, kurios nurodo ankstesnes išvestis, jų sunaudojimą, naujas išvestis ir grąžą. Pradines lėšas išreikšk pradiniais UTXO. Patikrink nežinomą ir pakartotinai išleidžiamą išvestį bei sumų išsaugojimą.
Lygiagretus kasimas v0.2: +0,5	Kandidatus kask gijomis ar procesais. Užtikrink nepriklausomą paiešką, saugų vieno laimėtojo priėmimą ir kitų darbuotojų sustabdymą. Palygink su nuosekliu vykdymu vienodomis sąlygomis.

Papildomi sprendimai neatstoja pagrindinės užduoties. UTXO variante pinigų išsaugojimą tikrink pagal neišleistų išvesčių sumą. Lygiagretus vykdymas viename kompiuteryje savaime nėra decentralizuotas tinklas.

Šaltiniai tolesniam skaitymui

- Bitcoin Developer Guide: Block Chain - blokų susiejimas, PoW ir Merkle medžiai.
- Bitcoin Developer Reference: Block Chain - realaus Bitcoin bloko struktūra palyginimui.

Riba: šiame darbe naudojama sava maiša, supaprastintas lėšų modelis ir vietinė vykdymo aplinka. Net sėkmingai atlikti testai neįrodo kriptografinio saugumo ar tinkamumo realioms lėšoms.